

“MAKALAH TUGAS 2 SISTEM OPRASI”

**Disusun Untuk Memenuhi Tugas Mata Kuliah
Sistem Oprasi**

Dosen : Triawan Adi Cahyanto, M.Kom



**Disusun Oleh:
Reandra Khansa Faadihilah 2410651045**

**UNIVERSITAS MUHAMMADIYAH JEMBER
2025-2026**

DAFTAR ISI

BAB I SISTEM MANAJEMEN PROSES	3
BAB II CHAT APPLICATION DENGAN IPC	5

BAB I SISTEM MANAJEMEN PROSES

Buat program C yang mensimulasikan sistem manajemen proses sederhana dengan fitur:

- Membuat multiple child processes
- Setiap process melakukan tugas berbeda (sorting, searching, calculation)
- Parent process mengkoordinasi dan mengumpulkan hasil
- Tampilkan PID, execution time, dan status setiap proses

Membuat file manajemen_proses.c dengan menggunakan gedit

```
^ ~ > ls @ 21:31
auto-cpufreq Clone Desktop Documents Downloads down-wallset
^ ~ > mkdir Tugas_S02 @ 21:31
^ ~ > ls @ 21:31
auto-cpufreq Clone Desktop Documents Downloads down-wallset
^ ~ > cd Tugas_S02/ @ 21:31
^ ~/Tugas_S02 > gedit manajemen_proses.c @ 21:33
```

```
manajemen_proses.c
~/Tugas_S02
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5 #include <time.h>
6
7 int main() {
8     printf("Sistem Manajemen Proses by Kansaa\n");
9
10    pid_t pid1, pid2, pid3;
11    clock_t start, end;
12    double time_taken;
13
14    start = clock();
15    pid1 = fork();
16    if (pid1 == 0) {
17        printf("\n[Child 1] PID: %d - Tugas: Sorting\n", getpid());
18        int data[5] = {5, 3, 1, 4, 2};
19        // Bubble sort sederhana
20        for (int i = 0; i < 5; i++) {
21            for (int j = 0; j < 4; j++) {
22                if (data[j] > data[j + 1]) {
23                    int temp = data[j];
24                    data[j] = data[j + 1];
25                    data[j + 1] = temp;
26                }
27            }
28        }
29        printf("[Child 1] Hasil sorting: ");
30        for (int i = 0; i < 5; i++) printf("%d ", data[i]);
31        printf("\n[Child 1] Selesai\n");
32        exit(0);
33    }
34
35    pid2 = fork();
36    if (pid2 == 0) {
37        printf("\n[Child 2] PID: %d - Tugas: Searching angka 7\n", getpid());
38        int data[5] = {2, 4, 6, 7, 9};
39        int target = 7;
40        int found = 0;
41        for (int i = 0; i < 5; i++) {
42
43            for (int i = 0; i < 5; i++) {
44                if (data[i] == target) {
45                    found = 1;
46                    break;
47                }
48            }
49            if (found)
50                printf("[Child 2] Angka %d ditemukan!\n", target);
51            else
52                printf("[Child 2] Angka %d tidak ditemukan.\n", target);
53            printf("[Child 2] Selesai\n");
54            exit(0);
55        }
56
57        pid3 = fork();
58        if (pid3 == 0) {
59            printf("\n[Child 3] PID: %d - Tugas: Kalkulasi sederhana\n", getpid());
60            int hasil = 0;
61            for (int i = 1; i <= 5; i++) {
62                hasil += i;
63            }
64            printf("[Child 3] Hasil penjumlahan 1+2+3+4+5 = %d\n", hasil);
65            printf("[Child 3] Selesai\n");
66            exit(0);
67        }
68
69        for (int i = 0; i < 3; i++) {
70            wait(NULL);
71        }
72
73        end = clock();
74        time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
75
76        printf("\n[Parent] Semua proses selesai!\n");
77        printf("[Parent] PID: %d\n", getpid());
78        printf("[Parent] Waktu eksekusi total: %.3f detik\n", time_taken);
79
80        return 0;
81    }
82 }
```

Compile dan jalankan program :

```
^ ~/Tugas_S02 > ./manajemen_proses
Sistem Manajemen Proses by Kansaa

[Child 2] PID: 27376 - Tugas: Searching angka 7
[Child 2] Angka 7 ditemukan!
[Child 2] Selesai!

[Child 1] PID: 27375 - Tugas: Sorting
[Child 1] Hasil sorting: 1 2 3 4 5
[Child 1] Selesai!

[Child 3] PID: 27377 - Tugas: Kalkulasi sederhana
[Child 3] Hasil penjumlahan 1+2+3+4+5 = 15
[Child 3] Selesai!

[Parent] Semua proses selesai!
[Parent] PID: 27374
[Parent] Waktu eksekusi total: 0.000 detik
^ ~/Tugas_S02 > |
```

Penjelasan:

a. Membuat multiple child processes

Program membuat tiga proses anak dengan memanggil `fork()` sebanyak tiga kali:

`pid1` → Child ke-1

`pid2` → Child ke-2

`pid3` → Child ke-3

Setiap anak memiliki PID dengan menggunakan `getpid()`

b. Setiap process melakukan tugas berbeda

- Child 1: Melakukan sorting (pengurutan angka) menggunakan algoritma bubble sort
- Child 2: Melakukan pencarian angka (searching) di dalam array.
- Child 3: Melakukan kalkulasi sederhana (penjumlahan $1+2+3+4+5$)

c. Parent process mengkoordinasi dan mengumpulkan hasil

Menggunakan `wait(NULL)` untuk menunggu setiap child process selesai menjalankan tugasnya

Karena Child ada 3 maka diulang sebanyak 3 kali :

```
for (int i = 0; i < 3; i++) {  
    wait(NULL);  
}
```

Setiap child diberi akhiran `exit(0)` guna memberi tahu jika child process selesai maka, bisa lanjut ke proses `wait()`

d. Menampilkan PID, execution time, dan status setiap proses

- Untuk menampilkan PID setiap proses dan status setiap proses menggunakan `printf()`
- Mengukur waktu program Ketika dijalankan dengan menggunakan `clock()` dan `CLOCKS_PER_SEC`.

Kesimpulan :

Program berhasil mensimulasikan sistem manajemen proses sederhana dengan tiga child process yang menjalankan tugas berbeda (sorting, searching, dan calculation). Parent process berfungsi mengkoordinasikan seluruh proses anak, menunggu hingga semua selesai, serta menampilkan waktu eksekusi total dan status setiap proses. Program ini menunjukkan bagaimana sistem operasi mengelola dan menjalankan proses secara paralel menggunakan konsep `fork` dan `wait`.

BAB II

CHAT APPLICATION DENGAN IPC

Implementasikan aplikasi chat sederhana menggunakan:

- Shared Memory atau Message Queue untuk menyimpan pesan
- Multiple processes sebagai pengguna
- Mutex/Semaphore untuk sinkronisasi
- Fitur: kirim pesan, lihat history, exit

Pertama membuat 3 file chat_basic.c, chat_f, chat_main dengan menggunakan gedit

chat_basic.c :

```
^ ~/Tugas_S02 > gedit chat_basic.c c -gcc @ 23:15
```

```
Open  ▾  [icon]  *chat_basic.c
~Tugas_S02

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <sys/ipc.h>
6 #include <sys/shm.h>
7 #include <sys/sem.h>
8
9 #define SHM_SIZE 2048
10 #define SEM_KEY 1234
11
12 struct chat_data {
13     char history[SHM_SIZE];
14 };
15
16 int create_shared_memory();
17 int create_semaphore();
18 void semaphore_wait(int semid);
19 void semaphore_signal(int semid);
20 void write_message(int shmid, const char *msg);
21 void read_messages(int shmid);
22
```

chat_f.c :

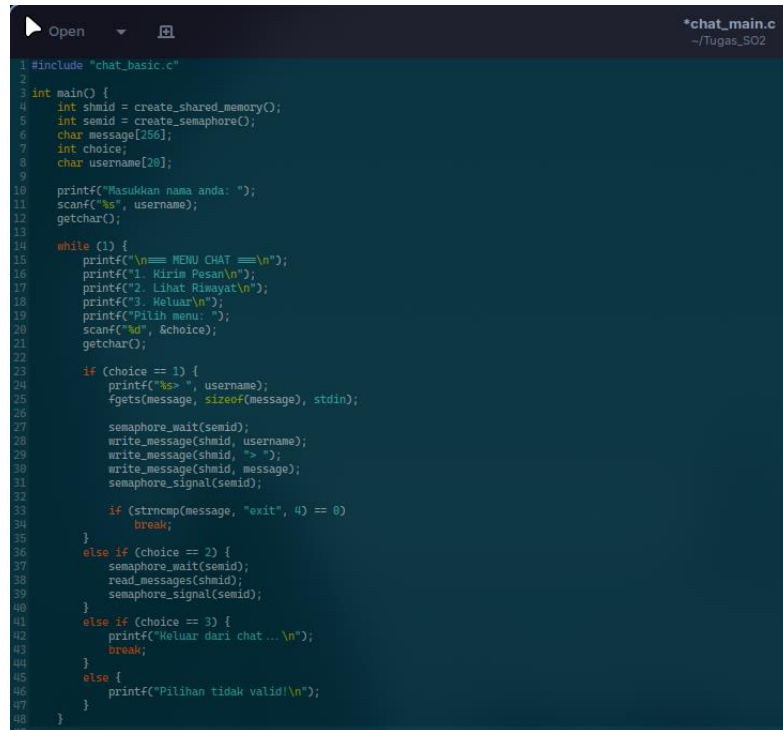
```
^ ~/Tugas_S02 > gedit chat_f.c c -gcc @ 23:24
```

```
Open  ▾  [icon]  *chat_f.c
~Tugas_S02

1 #include "chat_basic.c"
2
3 int create_shared_memory() {
4     key_t key = ftok("chat-file", 65);
5     int shmid = shmget(key, sizeof(struct chat_data), 0666 | IPC_CREAT);
6     if (shmid == -1) {
7         perror("Gagal membuat shared memory");
8         exit(1);
9     }
10    return shmid;
11 }
12
13 int create_semaphore() {
14     int semid = semget(SEM_KEY, 1, 0666 | IPC_CREAT);
15     semctl(semid, 0, SETVAL, 1);
16     return semid;
17 }
18
19 void semaphore_wait(int semid) {
20     struct sembuf sb = {0, -1, 0};
21     semop(semid, sb, 1);
22 }
23
24 void semaphore_signal(int semid) {
25     struct sembuf sb = {0, 1, 0};
26     semop(semid, sb, 1);
27 }
28
29 void write_message(int shmid, const char *msg) {
30     struct chat_data *data = (struct chat_data *)shmat(shmid, NULL, 0);
31     strcat(data->history, msg);
32     shmdt(data);
33 }
34
35 void read_messages(int shmid) {
36     struct chat_data *data = (struct chat_data *)shmat(shmid, NULL, 0);
37     printf("Rimayati chat --> %s\n", data->history);
38     shmdt(data);
39 }
```

chat_main.c :

```
^ ~/Tugas_S02 > gedit chat_main.c
```

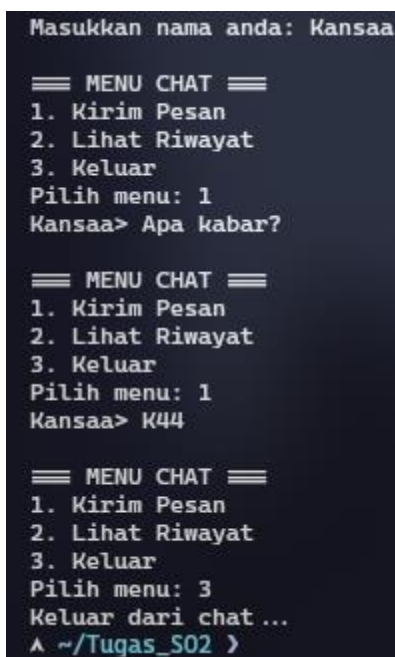


```
1 #include "chat_basic.c"
2
3 int main() {
4     int shmid = create_shared_memory();
5     int semid = create_semaphore();
6     char message[256];
7     int choice;
8     char username[20];
9
10    printf("Masukkan nama anda: ");
11    scanf("%s", username);
12    getchar();
13
14    while (1) {
15        printf("\n== MENU CHAT ==\n");
16        printf("1. Kirim Pesan\n");
17        printf("2. Lihat Riwayat\n");
18        printf("3. Keluar\n");
19        printf("Pilih menu: ");
20        scanf("%d", &choice);
21        getchar();
22
23        if (choice == 1) {
24            printf("%s> ", username);
25            fgets(message, sizeof(message), stdin);
26
27            semaphore_wait(semid);
28            write_message(shmid, username);
29            write_message(shmid, "> ");
30            write_message(shmid, message);
31            semaphore_signal(semid);
32
33            if (strcmp(message, "exit", 4) == 0)
34                break;
35
36            else if (choice == 2) {
37                semaphore_wait(semid);
38                read_messages(shmid);
39                semaphore_signal(semid);
40            }
41            else if (choice == 3) {
42                printf("Keluar dari chat...\n");
43                break;
44            }
45            else {
46                printf("Pilihan tidak valid!\n");
47            }
48        }
49    }
```

Compile dan jalankan program :

```
^ ~/Tugas_S02 > gcc chat_basic.c chat_f.c chat_main.c -o chat c -gcc © 23:39
^ ~/Tugas_S02 > ./chat c -gcc © 23:40
```

Terminal 1 :



```
Masukkan nama anda: Kansaa

== MENU CHAT ==
1. Kirim Pesan
2. Lihat Riwayat
3. Keluar
Pilih menu: 1
Kansaa> Apa kabar?

== MENU CHAT ==
1. Kirim Pesan
2. Lihat Riwayat
3. Keluar
Pilih menu: 1
Kansaa> K44

== MENU CHAT ==
1. Kirim Pesan
2. Lihat Riwayat
3. Keluar
Pilih menu: 3
Keluar dari chat...
^ ~/Tugas_S02 >
```

Terminal 2

```
Masukkan nama anda: Rens

== MENU CHAT ==
1. Kirim Pesan
2. Lihat Riwayat
3. Keluar
Pilih menu: 2

-- Riwayat Chat --
Kansaa> Apa kabar?
Kansaa> K44
```

Penjelasan

chat_basic.c :

Menyimpan semua deklarasi, struktur data, dan header library yang dibutuhkan

- #include berfungsi memanggil library c yang berada di sistem Linux
- #define SHM_SIZE 2048
- Fungsi create_shared_memory(), create_semaphore(), write_message, read message
Berfungsi untuk membuat shared memory, semaphore dan menulis pesan serta membaca pesan yang deklarasi di file lain

chat_f.c :

Menjalankan semua fungsi yang digunakan dalam program

- #include chat_basic.c guna menggunakan semua deklarasi dari file chat_basic.c
- int create_shared_memory()
Membuat atau mengambil shared memory yang bisa diakses beberapa proses.
menggunakan ftok() untuk membuat key unik, lalu shmget() untuk membuat segment memori.
- int create_semaphore()
Membuat semaphore dengan semget() dan mengatur nilai awalnya menjadi 1 (berarti tidak terkunci).
- void semaphore_wait(int semid)`
Fungsi (wait), digunakan sebelum proses menulis ke shared memory menurunkan nilai semaphore agar proses lain menunggu.
- void semaphore_signal(int semid)`
Digunakan setelah proses selesai menulis.
Menaikkan nilai semaphore agar proses lain bisa menulis

- `void write_message(int shmid, const char *msg)`
Menyimpan pesan ke dalam shared memory (menambahkan ke history)
- `void read_messages(int shmid)`
membaca pesan yang dikirim yang berada di history dari shared memory

chat_main.c :

Menjadi pusat program yang berinteraksi dengan user (antarmuka utama)

```
int shmid = create_shared_memory();
```

```
int semid = create_semaphore();
```

Membuat memori bersama dan semaphore untuk komunikasi

```
printf("Masukkan nama user: ");
```

```
scanf("%s", username);
```

Medeklarasi masukan dari pengguna dengan memasukkan nama

```
printf("\n=== MENU CHAT ===\n");
```

```
printf("1. Kirim Pesan\n");
```

```
printf("2. Lihat History\n");
```

```
printf("3. Exit\n");
```

(1) Kirim Pesan

- Meminta input pesan.
- Mengunci semaphore (`semaphore_wait`).
- Menulis pesan ke shared memory (`write_message`).
- Melepas kunci (`semaphore_signal`).

(2) Lihat History

- Menampilkan seluruh pesan dari shared memory.

(3) Exit

- Keluar dari program.

Kesimpulan :

Program chat IPC ini berhasil mengimplementasikan komunikasi antar proses menggunakan Shared Memory sebagai media berbagi pesan dan Semaphore sebagai mekanisme sinkronisasi. Melalui program ini, dua proses (user) dapat saling mengirim dan membaca pesan secara bergantian tanpa konflik akses. Dengan demikian, program memenuhi tujuan praktikum dalam memahami dan menerapkan konsep Interprocess Communication (IPC) pada sistem operasi Linux.